

Re-Architecting Traffic Control in Cross-Datacenter RDMA Networks

Zirui Wan¹, Jiao Zhang¹, Yuzhen Su¹, Haoyu Pan¹, Mingxuan Yu¹, Yuan Li¹,
and Tao Huang¹, *Senior Member, IEEE*

Abstract—The network-intensive applications, like machine learning and cloud storage, are increasingly driving two critical trends: 1) RDMA has been widely deployed to provide high-speed networks; 2) applications are distributively deployed across multiple regional datacenters to satisfy demands for content providers and customers. To fully utilize the benefits of RDMA, we desire to extend it to support cross-datacenter networks. However, the long-haul transport suffers a considerably long control loop, and thus the hybrid of long-haul and intra-datacenter traffic can easily cause severe congestion. We revisit existing traffic control methods and find they are insufficient to resolve this hybrid traffic congestion. Generally, regional datacenters are connected using dedicated long-haul optical fiber and datacenter interconnection (DCI) switches. In this paper, we propose Approach Traffic Control (ATC), a novel solution focusing on two-side DCI-switches (i.e., the approach point for datacenters) to separately alleviate the hybrid traffic congestion in the local and distal datacenters, as a building block for host-driven control methods. This design principle helps ATC shorten the control loop to a single datacenter scale while aggregating congestion information of the whole datacenter range with minor deployment complexity. We implement ATC on P4-based switches and conduct evaluations using real-world testbeds and large-scale NS3 simulations. The results show that ATC ensures fast congestion avoidance and delivers significant performance. For example, ATC reduces the FCT of intra-datacenter and long-haul traffic by up to 88% and 52%, respectively.

Index Terms—RDMA network, cross-datacenter network, congestion control.

Received 14 August 2024; revised 14 November 2024 and 18 April 2025; accepted 7 May 2025; approved by IEEE TRANSACTIONS ON NETWORKING Editor J. P. Jue. This work was supported in part by the National Key Research and Development Program of China under Grant 2022YFB2901700, in part by the National Natural Science Foundation of China (NSFC) under Grant 62132022, in part by Shandong Provincial Natural Science Foundation under Grant ZR2023LZH011, and in part by the Program for Youth Innovative Research Team of Beijing University of Posts and Telecommunications (BUPT) under Grant 2024YQTD02. (*Corresponding author: Jiao Zhang.*)

Zirui Wan, Haoyu Pan, Mingxuan Yu, and Yuan Li are with the State Key Laboratory of Networking and Switching Technology, Beijing University of Posts and Telecommunications, Beijing 100876, China.

Jiao Zhang and Tao Huang are with the State Key Laboratory of Networking and Switching Technology, Beijing University of Posts and Telecommunications, Beijing 100876, China, and also with Purple Mountain Laboratories, Nanjing 211111, China (e-mail: jiaozhang@bupt.edu.cn).

Yuzhen Su is with the State Key Laboratory of Networking and Switching Technology, Beijing University of Posts and Telecommunications, Beijing 100876, China, and also with China Telecom Research Institute, Beijing 102209, China.

Digital Object Identifier 10.1109/TON.2025.3569630

I. INTRODUCTION

REMOTE Direct Memory Access (RDMA) achieves high throughput and low latency with near-zero CPU overhead. With these advantages, RDMA is emerging as a popular high-speed networking technique that has been widely deployed in modern datacenters. For example, around 70% of traffic in Microsoft Azure is RDMA [1]. There is another trend that applications, like geo-distributed machine learning [2] and cloud storage [1], [3], are distributively deployed across multiple regional datacenters to satisfy demands for customers and content providers (e.g., Azure [1], Google [4], Facebook [5], Amazon [6] and Alibaba [3]). (§ II-A)

To fully utilize the benefits of RDMA, we desire to extend it to support cross-datacenter networks (§ II-B). The reasons are threefold. First, different from Wide Area Networks (WAN), regional datacenters are connected using dedicated long-haul optical fiber and DCI-switches (Figure 1) [7], [8]. This gives operators the ability to establish and manage RDMA transports over cross-datacenter networks. Second, RDMA can achieve 100Gbps throughput per-connection with almost zero CPU overhead while the TCP stack suffers low single-core throughput and continuous high CPU consumption, especially in the long-haul transport. Third, operators are reluctant to manage two networks and maintaining the usage of RDMA for cross-datacenter networks can avoid this problem.

However, designing a long-haul RDMA traffic control method faces challenges. Since the cross-datacenter network has a significantly larger base-RTT than the single datacenter scale, the long-haul transport suffers a considerably longer control loop. As a result, the hybrid of long-haul and intra-datacenter traffic complicates the network state and can easily cause severe congestion (i.e., hybrid traffic congestion). Meanwhile, the shallow buffer size of intra-datacenter switch is limited compared with the large Bandwidth Delay Product (BDP) of long-haul transport, which can easily trigger buffer overflow and result in PFC pauses.

Furthermore, we find that existing RDMA traffic control methods are mainly proper for the intra-datacenter networks and fail to address previous challenges (§ III-A). On the one hand, host-driven methods (e.g., DCQCN [9], TIMELY [10], HPCC [11]) suffer long control loops and fail to detect congestion in time. On the other hand, switch-driven methods [1], [12], [13], [14], [15] struggle to balance the trade-off between the difficulty of deployment and control range of

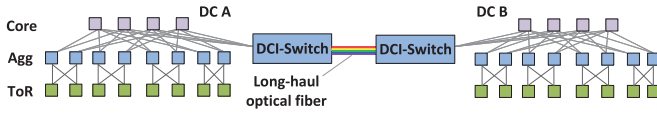


Fig. 1. Datacenter interconnect system.

congestion. Methods that need to deploy on overall switches increase deployment difficulty and tend to overreact to congestion (e.g., RoCC [13]) or put pressure on the shallow buffer of intra-datacenter switches (e.g., BFC [14]), while methods with partial switch deployment fail to deal with large-scale complicated congestion (e.g., Annulus [12]) or mainly focus on the long-haul link congestion (e.g., Azure [1], SWING [15]).

In this paper, we propose Approach Traffic Control (ATC), a novel solution focusing on two-side DCI-switches to separately alleviate the hybrid traffic congestion in the local and distal datacenters, as a building block for all existing host-driven control methods (§ IV). The design principle of the DCI-switch driven solution helps ATC shorten the control loop to a single datacenter scale while aggregating congestion information of the whole datacenter range with minor deployment complexity (e.g., ATC is compatible with RDMA NIC (RNIC) hardware-based DCQCN).

ATC is a readily deployable building block. We implement the DCI-switch pipeline of ATC on the P4-based switches and integrate it with host-driven control methods (§ V). We further build real-world testbeds together with large-scale NS3 simulations, to evaluate the performance of ATC (§ VI). The extensive results show that ATC ensures fast congestion avoidance and delivers significant performance. For instance, ATC reduces the FCT for intra-datacenter and inter-datacenter traffic by up to 88% and 52%, respectively.

Contributions. Our main contributions are:

- We **propose the requirement** to support long-haul RDMA transport. We **investigate the limitations** of existing traffic control methods: (i) host-driven methods suffer long control loops; (ii) switch-driven methods struggle to balance the trade-off between the difficulty of deployment and control range of congestion.
- We **present ATC, a novel long-haul RDMA traffic control solution** that focuses on two-side DCI-switches to separately alleviate hybrid traffic congestion in the local and distal datacenters, as a building block for all host-driven methods.
- We **implement ATC on commodity P4-based switches and conduct evaluations using both real-world testbed experiments and NS3 simulations.** The extensive evaluation results show that ATC is a promising substrate for strengthening the performance of cross-datacenter RDMA networks.

Note that despite ATC have been designed for RDMA network, we believe its insights and designs are also suitable for other high-speed cross-datacenter networking in general.

II. BACKGROUND

In this section, we first introduce the widely used deployment of cross-datacenter applications (§ II-A).

Then, we propose the need to provide long-haul RDMA support (§ II-B).

A. Cross-Datacenter Application Deployment

Multiple datacenters deployment. In recent years, high-performance applications are being distributively deployed across multiple datacenters and the reasons are twofold. First, a single datacenter has capacity limitations on computing and storage. Thus, large-scale data-intensive applications have been expanded across multiple datacenters to obtain sufficient resources [1], [7], [16]. Second, distributed applications satisfy demands for customers, including minimizing access latency (e.g., Content Delivery Networks (CDNs) [17], [18]), complying with data residency laws using on-premises data [19], [20], supporting regular data backup [21], [22], [23], [24] and provide better resilience to natural disasters.

Cross-datacenter traffic characteristic. In general, cross-datacenter traffic can be categorized into two types. The first type, called front-end user-generated traffic, derives from the immediate demand of users (e.g., video service, online chatting). The second type, called back-end business traffic, derives from the data processing or migration (e.g., geo-distributed machine learning system [2], web search system [25]). In recent years, the back-end traffic has increased significantly and poses new challenges to the performance of connectivity across datacenters [7], [21], [26].

Datacenter Interconnect (DCI) system. Large datacenter operators leverage dedicated long-haul optical fiber to connect their datacenters (e.g., Google B4 [26], Microsoft Global WAN [27], Facebook Backbone [28]). In order to satisfy the regional Service Level Agreement (SLAs) and the fiber restriction, the DCI system requires a dedicated fiber length of around 120 km and a round-trip latency of 1 ms between datacenters [8], [29], [30]. There are various choices for inter-datacenter connection and Figure 1 shows the common solution [7]. Each datacenter has an external DCI switch that connects to its core switches. Then, the DCI switch connects to the other side by the color optical transceiver and optical fiber [31]. These infrastructures enable Dense Wavelength Division Multiplexing (DWDM) to maximize the usefulness of fiber and increase bandwidth.

B. Why Long-Haul RDMA Is Needed?

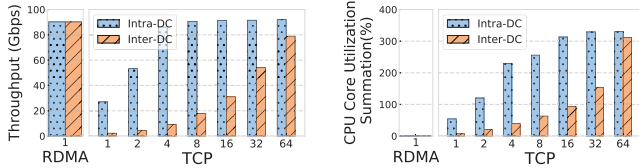
RDMA in the datacenter. RDMA is an innovative high-speed networking technique with kernel bypass and transport offload. RDMA can achieve high throughput and low latency with low CPU overhead. With these advantages, it becomes a trend to deploy RDMA to enhance underlying networks. Due to the high costs and closed architecture of Infiniband [32], another kind of RDMA technique, **RDMA over Converged Ethernet Version 2 (RoCEv2), is becoming the de-facto standard in modern high-speed datacenters** [3], [11], [33], [34].

Long-haul RoCEv2. To fully utilize the benefits of RDMA, we desire to extend it to support cross-datacenter networks. The reasons are twofold:

On the one hand, cross-datacenter applications like online streaming and data backup require high throughput to transmit

TABLE I
BUFFER SIZE OF COMMODITY INTRA-DC SWITCH AND DCI-SWITCH

	Intra-DC Switch			DCI-Switch	
Switch	Arista 7050	Cisco Nexus 9364	Mellanox SN4600	Cisco Nexus 9500R	Arista 7504R
Capacity (ports×BW)	32×100Gbps	64×100Gbps	64×100Gbps	36×400Gbps	96×400Gbps
Total buffer size	32MB	40MB	64MB	4GB	32GB
Buffer per port per Gbps	10KB	6.25KB	10KB	277KB	833KB



(a) Throughput under different flow number (b) CPU core utilization under different flow number

Fig. 2. Throughput and CPU core utilization under different flow numbers with TCP and RDMA transports in the intra-DC and inter-DC scenarios. RDMA can achieve high throughput with low CPU overhead in both scenarios.

large datasets. We conduct simple testbed experiments between two datacenters that are connected using a dedicated 100Gbps link (same as in § VI-A). As shown in Figure 2, RDMA provides a per-connection throughput of about 100Gbps with almost zero CPU overhead, while the bandwidth of traditional TCP is only tens of Gbps with unacceptable higher CPU overhead, especially in the cross-datacenter transport.

On the other hand, operators are reluctant to concurrently manage two networks [9], [35]. For instance, during RDMA/TCP hybrid deployment process, the highly I/O-intensive TCP will heavily consume PCIe bandwidth and provoke a large number of TX pauses (i.e., PFC pause sent by NICs) [3]. These TX pauses may result in PFC storms. Maintaining the usage of RDMA for cross-datacenter networks prevents datacenter operators and application developers from managing two kinds of networks (or programming interfaces).

III. MOTIVATION

In this section, we first analyze the requirements and challenges on rate control (§ III-A) for cross-datacenter RDMA networks. Then, we propose our design goals for the new long-haul RDMA traffic control method (§ III-B).

A. Existing Rate Controls Are Insufficient

Lossless RDMA. To enable efficient operation, RoCEv2 relies on PFC [36] to maintain a lossless (i.e., without congestion loss due to switch buffer overflow) network. With PFC, a switch sends a PAUSE message to the upstream entity when its ingress queue exceeds a certain threshold. The upstream entity stops sending packets till a RESUME message is received. PFC brings performance issues such as head-of-line (HoL) blocking, PFC storms and even deadlocks [9], [33], [37]. The fundamental solution for these issues is eliminating congestion by flow-level rate control methods.

However, different from intra-datacenter communications, the long-haul transport has following **three leading challenges**: (i) the base-RTT and BDP is significantly larger than

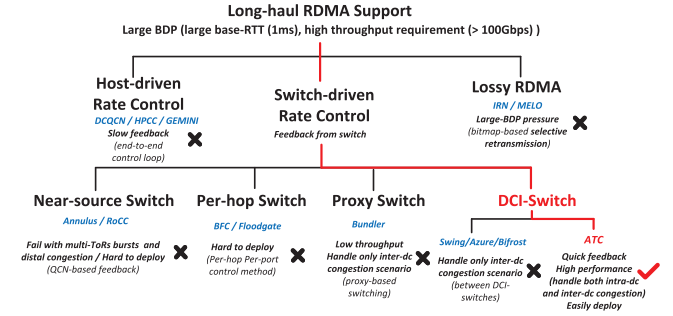


Fig. 3. The overview of existing control methods.

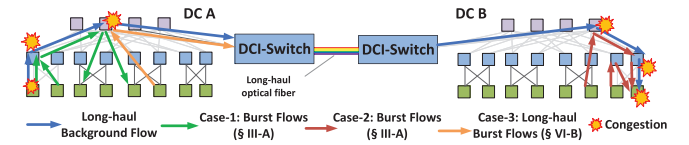


Fig. 4. The overview of hybrid traffic congestion.

single datacenter scale and thus the long-haul traffic suffers a long control loop; (ii) the hybrid of intra-datacenter and long-haul traffic complicates the network state and can easily cause severe congestion. We illustrate this in Figure 4. The **hybrid traffic congestion** happens when long-haul traffic bursts (Case-3) or conflicts with burst traffic from the local (Case-1) or distal (Case-2) intra-datacenter; (iii) the intra-datacenter pizza box switches have a limited shallow buffer [1] compared with the large BDP of long-haul transport as shown in Table I, which can easily trigger buffer overflow and result in PFC pauses. We find that existing rate control methods are insufficient to address these challenges as shown in Figure 3. We broadly categorize them as: (i) host-driven and (ii) switch-driven based on the entity playing the key role.

1) Host-driven rate controls suffer long control loops and fail to detect the risk of congestion in time. These control methods collect congestion signals (e.g., ECN in DCQCN [9], network delay in TIMELY [10], Swift [38], GEMINI [39], and INT in HPCC [11]) along the end-to-end loop. The long-haul transport considerably increases the feedback latency and make the sender fail to detect congestion in time. As a result, the long-haul traffic will converge to an error point. We consider the Case-1 and Case-2 of hybrid traffic congestion and show the performance issues. (We conduct experiments on the Case-3 in § VI-B.)

Mixture experiments for Case-1 and Case-2: We analyze the default deployed RNIC algorithm DCQCN and the state-of-the-art algorithm HPCC. We conduct NS3 experiments

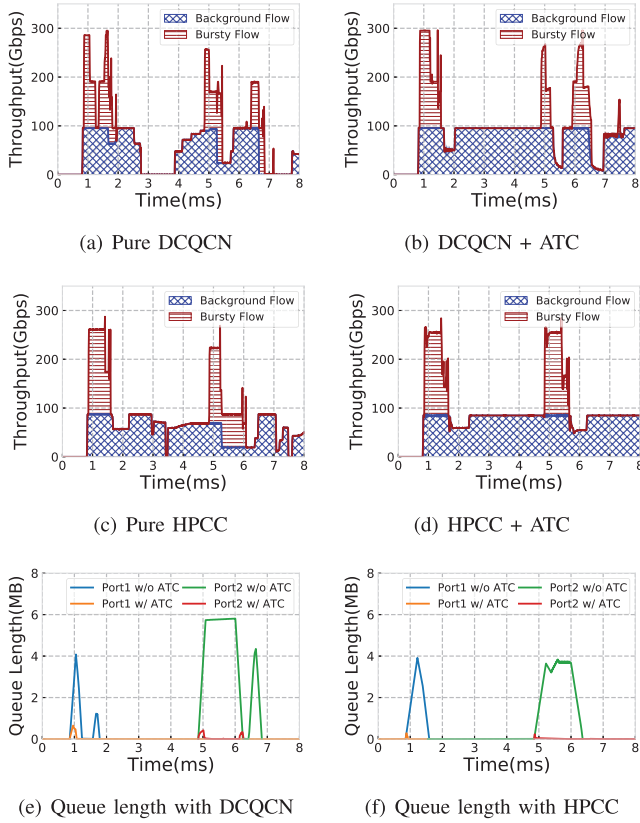


Fig. 5. Intra-DC burst flows start at time $0.8ms$ and $4.8ms$. Throughput and switch queue length when the long-haul traffic conflicts with intra-datacenter burst traffic. Port 1 and Port 2 are two congested ports in DC A and B. ATC can quickly reduce queue length and alleviate performance degradation.

and the setup is the same as in § VI-A. One long-lived background flow is sent from DC-A to one receiver in DC-B at time 0. Then, seven local intra-datacenter burst flows are transmitted from the same Pod with the background flow at time $0.8ms$ and another seven distal intra-datacenter burst flows are transmitted to the receiver within the same Pod with the background flow at time $4.8ms$. We conduct this bursty workload to evaluate the convergence of existing algorithms in burst scenarios by observing throughput variations of both background long-haul and burst flows. Figure 5 shows the throughput and switch queue length. We can see that the queue length peak reaches up to 6MB due to the long feedback delay, which triggers PFC and continually decrease the sending rate of both the intra-datacenter and long-haul traffic. As a result, the network converges to the error stage. At the worst possible time, the long-haul traffic throughput decreases to 0 and waits for a long increase interval to recover.

Then, we vary the configuration and conduct experiments using DCQCN with an increased sending window or HPCC with different window configurations (intra-datacenter or individual optimized¹). Figure 6 shows the results. We can see that on the one hand the increased sending window slightly improves the throughput. On the other hand, despite the performance of HPCC with individual optimized windows

¹HPCC with individual optimized windows denotes that different initial window are set for different traffic patterns.

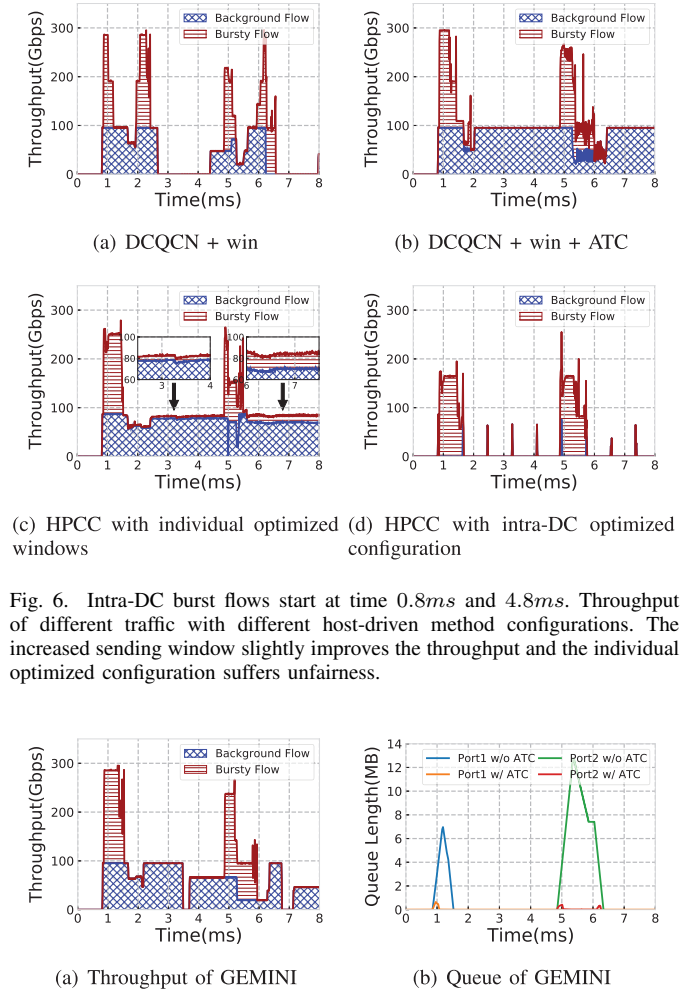


Fig. 6. Intra-DC burst flows start at time $0.8ms$ and $4.8ms$. Throughput of different traffic with different host-driven method configurations. The increased sending window slightly improves the throughput and the individual optimized configuration suffers unfairness.

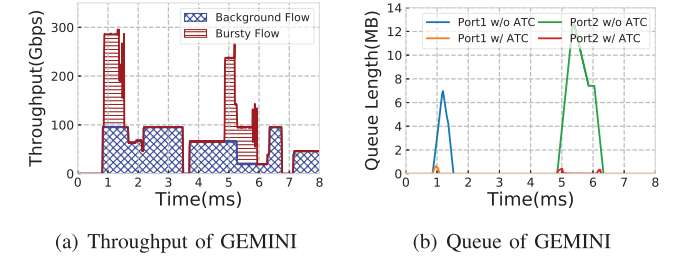


Fig. 7. Intra-DC burst flows start at time $0.8ms$ and $4.8ms$. Throughput and queue length of GEMINI. The long end-to-end control loop and inefficiencies in delay-based signaling for long-haul traffic still prevent GEMINI from mitigating congestion promptly.

being better, it suffers unfairness between the long-haul traffic and burst traffic. This is because the RTT and initial sending window of the two traffic are different and the additive increase factor W_{AI} fails in the long-haul scenario. As a result, the burst traffic suffers a long tail FCT. Besides, the intra-datacenter optimized configuration suffers severe long-haul traffic performance penalty. These results demonstrate that the root cause of long feedback delay leads to performance degradation for host-driven rate controls, regardless of how the rate adjustments are implemented.

We also evaluate the performance of GEMINI and Figure 7 shows the throughput and switch queue length. Due to GEMINI's long end-to-end control loop and the inefficiency of delay-based congestion signaling for long-haul traffic, the throughput convergence remains slow and the queue length peak reaches up to 12MB which triggers PFC.

2) Switch-driven rate controls struggle to balance the trade-off between the difficulty of deployment and control range of congestion. We broadly categorize them as: (i) near-source switch feedback solution, (ii) per-hop switch control and (iii) proxy switch control based on the location of the key

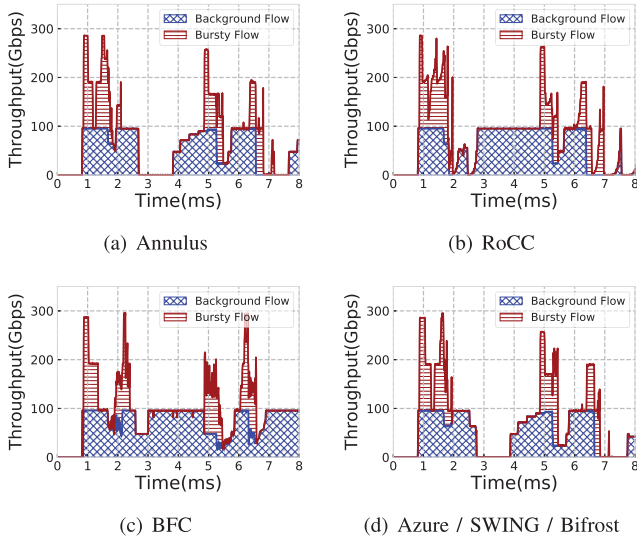


Fig. 8. Intra-DC burst flows start at time $0.8ms$ and $4.8ms$. Throughput of different traffic under different switch-driven rate controls. Annulus deploys only on ToRs but has a poor performance. RoCC and BFC have better performance but require deployment on all switches. Azure and SWING still maintain long control loops and have similar results with pure DCQCN when dealing with hybrid intra-DC congestion. In contrast, ATC ensure high performance and is easy to deploy.

switches. We repeat previous experiments with switch-driven rate controls and show the results in Figure 8.

Near-source switch feedback solutions directly send feedback to senders from switches (e.g., Annulus [12], RoCC [13]). Annulus deploys Quantized Congestion Notification (QCN) [40] standard only on ToRs to resolve intra-ToR bursts congestion. Thus, Annulus fails to resolve multi-ToRs bursts and the long-haul traffic still maintains long control loop in the distal hybrid congestion scenario. Figure 8(a) shows that Annulus only has a minor improvement compared to pure DCQCN. On the flip side, RoCC enables a QCN-like mechanism on every switch in the network, which is difficult for commercial datacenter deployment. In addition, Figure 8(b) shows that RoCC lacks global judgment of congestion and tend to overreact to both the background flow and bursty flows. RoCC also fails to resolve the congestion in the distal datacenter due to the long sender-to-distal control loop.

Per-hop switch control solutions provide per-hop per-port flow control (e.g., BFC [14], Floodgate [41]). These solutions maintain multiple queues in every switch to absorb burst packets and resolve congestion by per-hop credits control. However, the per-hop control adds difficulty to deployment. Figure 8(c) shows that BFC still suffers performance degradation for the background flow due to backpressure from shallow buffer intra-datacenter switches. Under heavy traffic loads, the long-haul traffic congestion will spread across intra-datacenter switches and put pressure on the number of queue usage, which increases hash collision and leads to the HoL problem. We conduct experiments to illustrate this viewpoint with intra-datacenter traffic at 30% load and inter-datacenter traffic at 30% load. We monitor the maximum number of VOQ usage in switches and Table II shows the results. We can see that

TABLE II
MAX USED VOQ NUMBER IN SWITCHES WITH BFC

Switch Number #	0	1	2	3	4	5	6	7	8	9	10	11
MAX number of VOQ usage	37	30	39	33	35	36	36	32	30	29	30	30
Switch Number #	12	13	14	15	16	17	18	19	20	21	22	23
MAX number of VOQ usage	28	32	28	28	32	29	37	30	33	31	34	28

the maximum number reaches 39 at #2 switch. In contrast, the maximum number in ATC is only 7 (Detailed in § VI-C).

Proxy switch control solutions use proxy switches to interconnect datacenters (e.g., Bundler [42]). These solutions mainly focus on the long-haul link congestion by aggregating and storing long-haul traffic into middleboxes on either side. Then, scheduling policies are enforced across the traffic between the middleboxes. The switching structure of proxies severely degrades the long-haul link throughput to only a few hundred Mbps [42]. With a similar idea, recent work SWING [15] leverages PFC-Relay between DCI-switches to ensure the throughput of long-distance links, Bifrost [43] preemptively pauses the local DCI-switch to reduce buffer usage and Azure [1] leverages off-chip DRAM on DCI-switches to store RDMA packets. However, these solutions also mainly focus on the long-haul link congestion and still maintain long control loops that fail to resolve the hybrid traffic congestion in both the local and distal datacenter. Thus, as shown in Figure 8(d), we note that Azure, SWING and Bifrost show similar performance with pure DCQCN.

Lossy RDMA. In recent years, many researches [34], [44], [45], [46], [47], [48], [49] advocate lossy RDMA that eliminates PFC, by replacing go-back-N mechanism with efficient bitmap-based selective repeat (SR). In the long-haul transport scenario, the SR mechanism can reduce the performance impacts caused by shallow buffer datacenter switches and consequential PFC pause frames spreading. However, a production-level comprehensive PFC-free solution is still premature [3], and the large long-haul BDP puts pressure on their bits pool and fixed bitmap size configuration. In this paper, we primarily focus on the rate control scheme that is orthogonal and complementary with these lossy RDMA efforts. We believe the development of lossy RDMA will improve the long-haul transport.

B. Lessons Learned and Design Goals

We advocate our **design goals** for the new long-haul RDMA traffic control method:

1) **Fast congestion avoidance.** The new solution should shorten the control loop to detect congestion in time.

2) **Large congestion control range while ensuring high performance.** The new solution should maintain the congestion control range across the whole datacenter to deal with large-scale complicated congestion. Meanwhile, the new solution should be compatible with intra-datacenter traffic controls to ensure high performance.

3) **Easy to deploy in commodity datacenters.** The new solution should be simple enough to make minimal

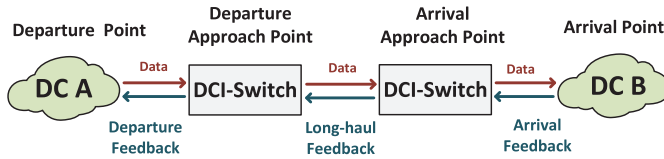


Fig. 9. The overview of ATC framework.

incremental changes to the existing production datacenters and be feasible with commodity entities (e.g., RNIC and shallow-buffered intra-datacenter switch hardware).

Strawman proposal for approach control. An intuitive idea is to separately divide the end-to-end control loop into three distinct controls, relying on two-side DCI-switches (i.e., the approach point for datacenters) to separately control the hybrid traffic congestion in the local and distal datacenter while delegating the pure intra-datacenter congestion to the host-driven control method. This has three major benefits: (i) The control loop is shortened to the scale of a single datacenter; (ii) The DCI-switch can aggregate relative link load information of the long-haul traffic across the whole datacenter by monitoring congestion signals, such as detecting ECN marks, recording the timestamp and calculating the RTT, and tracking the INT information, aligning with the deployed host-driven methods. These capabilities are supported by modern commodity programmable switches. Thus, the DCI-switch provides a greater range of control; (iii) This solution is easy to deploy in production datacenters.

ATC brief glance. We repeat the previous experiments with ATC and find that ATC can quickly reduce queue length by up to 94% and thus ensure consistent high performance of both long-haul traffic and intra-datacenter burst flows. ATC also performs better than other switch-driven rate controls.

IV. ATC DESIGN

In this section, we propose Approach Traffic Control (ATC).² The key design choice of ATC is to rely on DCI-switches to support the long-haul RDMA transport while maintaining high intra-datacenter performance by cooperating with host-driven control methods. As shown in Figure 9 and Algorithm 1, ATC is composed of four points: departure point (local datacenter), departure/arrival approach point (i.e., local/distal DCI-switch), and arrival point (distal datacenter). The core logic of ATC mainly comprises three main traffic control modules, named departure control, long-haul control, and arrival control, positioned sequentially located across the previous four points:

- **Departure Traffic Control:** from local DCI-switch to local datacenter. To promptly reconcile hybrid traffic congestion in the local datacenter, the local DCI-switch directly generates departure feedback with congestion information back to the sender, which significantly

shortens the control loop of local hybrid traffic congestion. Then, the sender can adjust its sending rate in time. (§ IV-A)

- **Arrival Traffic Control:** from distal datacenter to distal DCI-switch. To resolve the hybrid traffic congestion in the distal datacenter, ATC introduces a novel flow scheduling mechanism at the distal DCI-switch, which schedules the transmission of data packets based on the arrival feedback generated by the receiver. With such a scheme, ATC can shorten the control loop of distal hybrid traffic congestion and efficiently converge without modifying distal datacenter entities. In addition, ATC leverages Virtual Output Queue (VOQ) to prevent the HoL problem. (§ IV-A)

Algorithm 1 Approach Traffic Control (ATC) Algorithm

```

1: function DEPARTURE_TRAFFIC_CONTROL(Pkt)
2:   Monitor_Congestion_Signal(Pkt)
3:   Generate_Departure_Feedback()
4: end function
5: function LONG-HAUL_TRAFFIC_CONTROL(Pkt)
6:   Forward_Scheduler(Pkt)
7: end function
8: function ARRIVAL_TRAFFIC_CONTROL(Pkt, Feedback)
9:   VOQ_Allocation(Feedback)
10:  VOQ_Scheduler(Feedback)
11:  Forward_Packet(Pkt)
12: end function
13: procedure LOCAL DCI-SWITCH(Pkt)
14:   DEPARTURE_TRAFFIC_CONTROL(Pkt)
15:   LONG-HAUL_TRAFFIC_CONTROL(Pkt)
16: end procedure
17: procedure DISTAL DCI-SWITCH(Pkt, Feedback)
18:   ARRIVAL_TRAFFIC_CONTROL(Pkt, Feedback)
19: end procedure

```

- **Long-haul Traffic Control:** from distal DCI-switch to local DCI-switch. To further reduce the buffer usage and number of VOQ usage in DCI-switches, ATC introduces the long-haul feedback mechanism to proactively schedule packets transmission between DCI-switches and satisfy insufficient VOQ scenarios. (§ IV-B)

Main challenges in ATC. There are three major challenges to realizing design choices: (i) the control logic should be simple enough to deploy in production datacenters; (ii) existing host-driven controls have variable feedback and rate control methods. ATC should be compatible with these mechanisms; (iii) how to allocate VOQ resources with high efficiency and scalability to satisfy complex long-haul scenarios.

A. Departure and Arrival Traffic Control

Departure traffic control logic. To reconcile hybrid traffic congestion in the local datacenter, the local DCI-switch directly generates departure feedback to the sender when it detects congestion has happened (including congestion at the local DCI-switch). To ensure fairness and simplify deployment difficulty, ATC leverages the same congestion feedback as the deployed host-driven method. For example,

²This work is inspired by the Air Traffic Control (ATC) system that provides orderly and efficient services to the aircraft in the air and on the ground. In ATC system, the approach service (located in the control tower) plays a key role in providing guidance to the aircraft for taking off or landing. We liken cross-datacenter traffic to the aircraft due to their long-haul transport nature and deploy the approach service in the DCI-switches.

when using DCQCN, ATC generates Congestion Notification Packets (CNP) if the data packet is ECN-marked. When using HPCC, ATC cuts off the INT headers and mirrors them back to the sender. Then, after receiving departure feedback, the sender will update the transmitting rate using the same adjustment algorithm as the deployed host-driven method.

Arrival traffic control logic. The distal DCI-switch firstly clears the congestion markers (e.g., ECN-marked, INT header) in the data packet to reduce interference from other congestion signals and save link bandwidth. Then, the distal DCI-switch leverages the following two mechanisms to reconcile hybrid traffic congestion in the distal datacenter:

1) Schedule transmission in a per-destination aggregated manner. To reduce deployment complexity and enhance scalability, the distal DCI-switch restricts packets transmission by maintaining a per-destination aggregated *Schedule Table*. Each entry in *Schedule Table* includes the destination server information and the relative transmission restriction, and thus shares the same keys and scale as the *Forwarding Table* and differs in their respective values. Specifically, the transmission restriction is the rate (e.g., forwarding rate) or window (e.g., inflight bytes between the distal DCI-switch and receiver) based on the deployed host-driven method. When the distal DCI-switch receives a data packet, it checks the *Schedule Table* and determines whether the packet can be forwarded. To calculate transmission restriction, ATC utilizes ACKs for arrival feedback, since most host-driven methods carry congestion information in ACKs. Then, after receiving arrival feedback, the distal DCI-switch calculates the risk of congestion in the distal datacenter and updates the *Schedule Table* using the same adjustment algorithm as the deployed host-driven method, thereby avoiding overreaction. Last, the distal DCI-switch clears the congestion markers (e.g., INT header) in the arrival feedback.

2) Avoid the HoL problem using dynamically VOQs. To avoid the HoL problem at the distal DCI-switch, ATC leverages dynamically allocated VOQs to aggregate and store data packets with the same destination server. Each VOQ corresponds to one destination server. As show in Table I, the DCI-switch adopts deep buffers using multiple switch ASICs based on the VOQ architecture [1]. A VOQ is a collection of buffers that store traffic destined to one egress port [50], [51], [52], [53]. The VOQ does not transmit packets until the egress port has the resource to forward. Some switch chips like Jericho [54] and Silicon One [55] can support 100K VOQ queues [56], which is sufficient for common production datacenters.

In ATC, when the distal DCI-switch receives a data packet, it checks whether the destination has been allocated with one VOQ. If so, it indicates that hybrid traffic congestion has happened in the distal datacenter and the data packet will be pushed into the corresponding VOQ. Otherwise, the distal DCI-switch checks the *Schedule Table* whether the packet can be forwarded. If not, the distal DCI-switch will allocate an empty VOQ for the data packet based on the VOQ bitmap. Otherwise, the distal DCI-switch directly forwards it.

Compatible with window-based host-driven adjustment. When the deployed host-driven method is a window-based

scheme (e.g., HPCC), ATC leverages the following two mechanisms to cooperate with them:

First, the sender in ATC choose to limit the inflight bytes between the sender and local DCI-switch. Benefiting from this, the sender does not need to distinguish between long-haul and intra-datacenter traffic and can directly assign their initial window to the intra-datacenter BDP. At the local DCI-switch, when data packets are forwarded, departure feedback is generated with the corresponding data packet size. Then, after receiving departure feedback, the sender updates the transmitting window and controls inflight bytes using the same adjustment algorithm as the deployed host-driven method.

Second, the distal DCI-switch limits the inflight bytes between the distal DCI-switch and the receiver and assigns the initial window size to the intra-datacenter BDP. The receiver inserts the received data packet size to the corresponding arrival feedback. Then, after receiving arrival feedback, the distal DCI-switch calculates the inflight bytes and updates the transmission restriction.

Compatible with load balance mechanism in the distal datacenter. ATC schedules traffic in a per-destination aggregated manner at the distal DCI-switch. However, under the Equal-cost multi-path (ECMP) load balance mechanism, traffic with different *5-tuple* may pass through different links in the distal datacenter and obtain incorrect congestion information.

To address this problem while ensuring that intra-datacenter traffic continues to benefit from ECMP-based load balancing, there are two mechanisms: (i) *Specific ECMP module*. Some commodity switches have the destination-based ECMP routing module, which ensure the same path for the per-destination aggregated traffic [11]; (ii) *ECMP spoofing*. The distal DCI-switch inserts spoofed headers to the data packet and thus ECMP-based routing produces the same result for traffic with the same destination [57]. When supported by the production datacenter, ATC chooses the first mechanism. Otherwise, the second mechanism is used. In addition, since ACKs have the highest priority and data centers typically use a FatTree topology with a symmetrical structure, ACKs from the same receiver ensure the arrival order at the distal DCI-switch. Thus, the distal DCI-switch can resolve congestion information in sequence.

Restrict VOQ queue length to reduce tail latency. The VOQ collects traffic with the same destination when congestion happens. However, the continuous accumulated packets in the VOQ will block the new incoming traffic and increase the tail latency. To further restrict VOQ queue length, the distal DCI-switch inserts the corresponding transmission restriction to ACKs. Then, after receiving ACKs, the sender will schedule the transmission by choosing the more restricted one between the departure feedback restriction and distal DCI-switch restriction. To reduce deployment complexity with widely deployed DCQCN, ATC partially forwards CNPs at the distal DCI-switch as an alternative to modifying ACKs.

B. Long-Haul Traffic Control

Limit the number of VOQ usage. Note that in some production datacenters, the number of VOQs may be smaller than

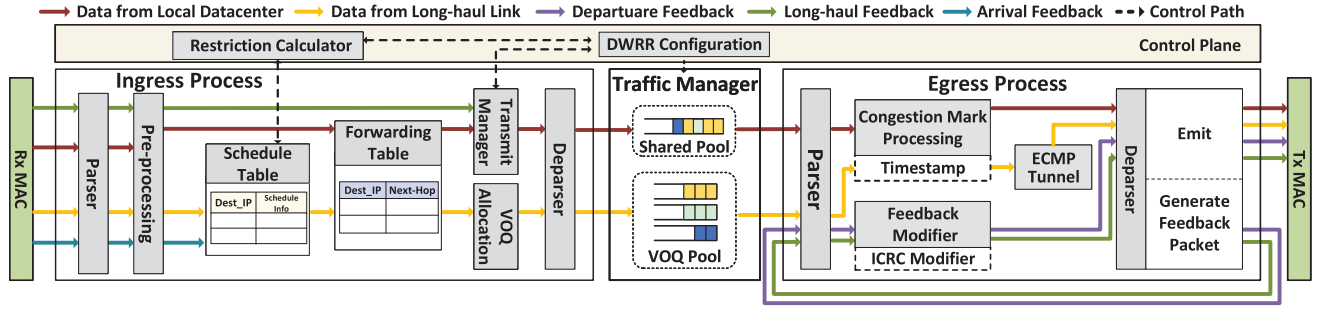


Fig. 10. P4-based Switch Pipeline of DCI-switches in ATC.

the number of destination servers. In this insufficient VOQ case, ATC uses the following two mechanisms to constrain the usage of VOQ:

1) Proactively schedule packet transmission between DCI-switches using long-haul feedback. ATC controls the number of long-haul inflight bytes at the local DCI-switch to constrain the number of VOQ usage. The local DCI-switch records the total size of packets forwarded to the long-haul link. The distal DCI-switch records the total size of packets forwarded to the distal datacenter and generates long-haul feedback with this value every T period. After receiving long-haul feedback, the local DCI-switch calculates the inflight bytes and schedules the consequence transmission. Note that at the distal DCI-switch, when the remaining size in a VOQ is less than one MTU, corresponding packets will be forwarded directly to fully utilize the long-haul link.

2) Additional transmission restriction when VOQs are heavily utilized. When the number of active VOQ usage exceeds $N_{threshold}$, subsequent long-haul feedback will be marked. After receiving these marked feedback, the local DCI-switch will restrict the data packet forwarding rate by $K \times C_{longhaul}$, where K is the restriction factor and $C_{longhaul}$ is the bandwidth of long-haul link. To fully utilize the long-haul link, the initial number of inflight bytes is assigned to be $M \times BDP_{longhaul} + C_{longhaul} \times T$, where M is the bandwidth protection factor and $BDP_{longhaul}$ is the BDP of long-haul link. We discuss the parameter of this additional restriction mechanism (see online Appendix [58] for detailed discussion) and give our recommended values: $M = 2$, $K = \frac{1}{N_{total_VOQ_number}}$ and $N_{threshold} = \frac{2}{3} \times N_{total_VOQ_number}$.

Design tradeoff. In § VI, we conduct extensive experiments and the results show that ATC can perform well with only a dynamically allocate VOQ scheme in most cases (§ VI-D). However, we still maintain the proactively schedule scheme (i.e., long-haul traffic control) in this section to avoid corner cases and make ATC deployable with commodity switches due to the limited buffer usage of DCI-switches.

VOQ pre-allocated. In the considerable large-scale data-center, we advocate using the pre-allocated VOQ configuration to mitigate the insufficient VOQ problem. Some datacenters disaggregate compute and storage resources for cost savings [1]. Since the storage traffic has incast-like characteristics (e.g., stream layer data reconstruction), VOQs are pre-allocated and fixed to destination servers in the storage cluster.

V. IMPLEMENTATION

DCI-switch implementation. As shown in Figure 10, the ATC DCI-switch pipeline is readily deployable with the Tofino Native Architecture (TNA) [59]. The packet first traverses the ingress process, followed by the traffic manager (TM) and finally the egress process. ATC uses long-haul traffic control to limit the DCI-switch buffer usage and Tofino2 delivers up to 12.8 Tbps throughput and supports up to 400GbE port speed that meets the requirement of inter-datacenter interconnect [8], [29]. In addition, each port group has 128 queues with Tofino2 (totally with $128 \times$ port group number) that makes ATC deployable with production datacenters. Then, we introduce essential components:

Packet pre-processing module. This module has two roles: (i) distinguish between data packets and feedback packets by the ingress port and packet header. Then, the special header metadata is marked to identify different packets; (ii) at the arrival DCI-switch, the congestion markers carried in the packet header are cleared.

Schedule Table modifier module. After receiving arrival notification packets, the corresponding Schedule Table will be updated. To reduce the interaction frequency between the control plane and the data plane, we pre-process the congestion signal on the data plane and record it in a few Register. For example, as for TIMELY, we calculate the difference between timestamps in the data plane and record this calculated RTT. In addition, each Register has up to 10K entries on the switch ASIC, which allows ATC to be deployed in large-scale datacenters.

DWRR configuration module. Deficit Weighted Round Robin (DWRR) is a message bytes based scheduling algorithm used by the TM scheduler. Compared with Weighted Round Robin (WRR), DWRR can still allocate bandwidth resources by weight when the length of each data packet is unequal. After receiving feedback packets, the control plane will calculate the new weight and modify the scheduling configuration table (TNA component).

Congestion mark processing module. To address congestion at the local DCI-switch, it pushes data packets from the local datacenter to the congestion mark processing module in the egress process. To be compatible with TIMELY and address congestion in the distal data-center, the arrival DCI-switch marks data packets with the

timestamp in the egress process and marks corresponding arrival feedback packets with the timestamp in the ingress process.

Feedback packet generation module. To ensure high speed generation, we use *Mirror* feature in switch ASIC. Data packets are mirrored at the egress (allocate mirror session and truncate the packet) and will be modified later. We build the relations between QP pairs in the register and modify the destination QP number in the RDMA BTH header. Data packets and mirrored packets are distinguished using *metadata*.

In addition, Tofino2 supports programmable feedback packets generation mechanism that allows any ingress packet to trigger, which helps generate CNPs at the local DCI-switch and simplify the data plane.

ICRC modifier module. The ICRC field is mandatory for the RoCE packets to be accepted by the network adapter. ATC leverages the “Hash CRC32” function in Tofino ASIC to calculate ICRC based on the formula in [60] and [61] and adds it at the end of notification packets. Note that ATC only requires to calculate ICRC for the feedback packets (e.g., CNP) which only have a small amount of payload.

End-host implementation. ATC is directly compatible with RNIC hardware-based DCQCN. As for other host-driven control methods, ATC requires executing congestion control algorithms when receiving feedback packets. Fortunately, the new RNIC shows flexibly programmable congestion control logic. For instance, NVIDIA introduces Programmable Congestion Control (PCC) [62] functionality with their DPUs (e.g., BlueField3 [63]). The PCC framework collects congestion control events (e.g., timestamp) and supports self-defines control logic, facilitating the deployment of ATC alongside other methods that require modification of host-side control logic. We plan to implement ATC with these methods in the future.

VI. PERFORMANCE EVALUATION

We evaluate ATC using both real-world testbed experiments and large-scale simulations. The key findings are:

- **ATC is compatible with RNIC hardware-based DCQCN in the real-world testbed** with stable and high performance, e.g., reducing intra-datacenter FCT by up to 58%.
- **ATC ensures fast congestion avoidance compared with host-driven methods** with PFCs reduced by up to 98% and the overall average FCT improved by up to 75%.
- **ATC is easy to deploy while providing large congestion control range compared with switch-driven methods** with the 99th FCT reduced by up to 38%.

A. Evaluation Setup

Real-world testbed setup. As shown in Figure 11, we build two testbeds between datacenters that are connected using a dedicated 100Gbps link with a one-way distance of 100KM and 600KM, respectively. Unless stated otherwise, we mainly conduct evaluations on the 100KM testbed based on real production regional datacenters settings. DCI-switches and ToRs are connected via 100Gbps links. Each server has one NVIDIA 100Gbps RNIC connected to the ToR switch.

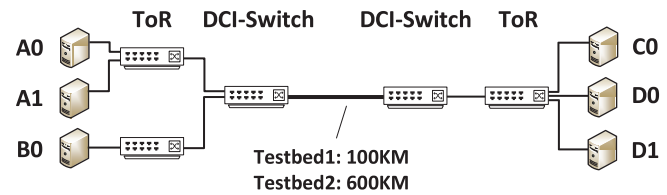


Fig. 11. Real-world testbed topology. We build two testbeds with 100KM and 600KM interconnect, respectively.

The default RNIC hardware congestion control algorithm is DCQCN. We deploy ATC on the testbed based on the § V. Our DCI-switch pipeline implementation consists of about 700 lines of P4 code and 300 lines of configurations via Barefoot Runtime (bfrt) APIs in the control plane. Since we do not have RNIC hardware based TIMELY and HPCC, we deploy them using DPDK. In addition, we are collaborating with one switch vendor to implement ATC to the switch hardware.

Simulation topology. In NS3 simulations, we use the same topology as in Figure 1. Both datacenters are built with FatTree [64] topologies. Each ToR switch is connected to four servers. The capacity of each link between two DCI-switch is 400Gbps and the bandwidth of other links are 100Gbps. All links within datacenters have a $1\mu s$ propagation delay and the propagation delay of the long-haul link is $400\mu s$ with 80km distance. The buffer size of switch within datacenter and DCI-switch are 16MB and 128MB, respectively. (Derived from Table I). To explore the usage of VOQ, we then move to a larger datacenter scale by changing the number of core, agg, ToR switches and servers to 16, 20, 20 and 320, respectively.

Parameter settings. For DCQCN, we use parameters suggested by a major NIC vendor as default [11] (the same as in HPCC simulations [65]). We scale the ECN marking threshold proportional to the link bandwidth the same as in [11]. For TIMELY, we use the parameters suggested in [10]. Following [11], a long-haul RTT based sending window is added to DCQCN and TIMELY to limit the inflight bytes. These mechanisms are called DCQCN+w and TIMELY+w. For HPCC, we set the initial window to the long-haul BDP by default to protect throughput and use the suggestions in [11] for the other parameters. We have evaluated Swift [38] and found its performance comparable to TIMELY+w under our hybrid traffic scenarios, and thus we omit its results for brevity. For other switch-driven control methods, we follow the suggestions in their papers (e.g., Annulus [12], RoCC [13], BFC [14], Azure [1], SWING [15]). We conduct experiments to evaluate the maximum number of VOQ usage and find that it is deployable with common switches. Thus, we consider the VOQ resources to be relatively adequate for ATC.

Workloads. We create some simple artificial workloads to evaluate the micro-benchmarks. In our large-scale simulations, we evaluate distributed storage scenario and use the traffic trace *AliStorage* [66], which is collected from a production distributed storage system and is more similar to real cross-datacenter traffic than other traffic traces [3], [46]. We adjust flow generation rates to set average intra-datacenter traffic and inter-datacenter traffic load between 20% and 50% based on

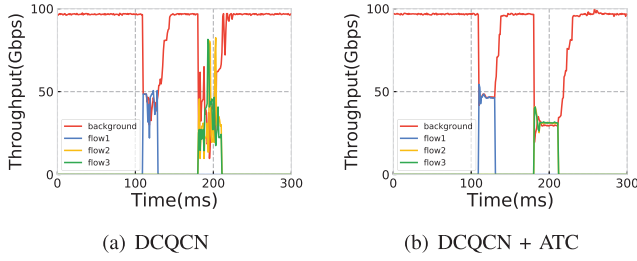


Fig. 12. [Testbed] Throughput of different flows when the long-haul traffic bursts (flow1) or conflicts with distal intra-datacenter burst traffic (flow2 and flow3).

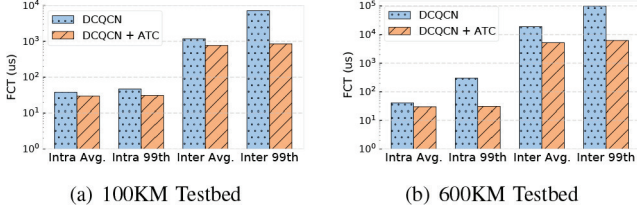


Fig. 13. [Testbed] FCT of different traffic with bursty workload in two testbeds. We generate long-haul burst flows and intra-datacenter burst flows.

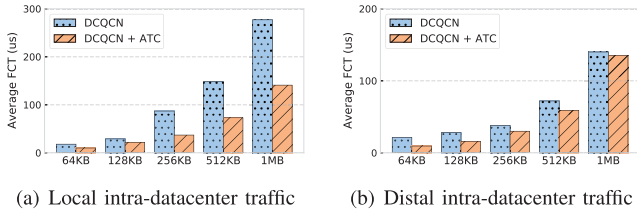


Fig. 14. [Testbed] Average FCT of intra-datacenter traffic with varying message sizes.

the NIC bandwidth. To make a stress test, we generate the inter-datacenter incast traffic at 20% load.

Performance metrics. We have four major metrics: (i) average/99th-tail FCTs; (ii) real-time bandwidth of individual flows; (iii) real-time network queues; (iv) PFC pause duration.

B. Real-World Testbed Experiments

ATC ensures fast convergence. We generate one long-haul background flow from host A0 to C0 and add another long-haul burst flow from host A1 at 100ms following two distal burst flows from host group D to C0. Figure 12 shows the results. We can see that ATC has faster congestion avoidance and rate convergence benefiting from its short control loop.

ATC strengths end-to-end performance. We generate long-haul and intra-datacenter burst traffic under 100KM and 600KM testbeds. Figure 13 shows the results. We can see that ATC reduces 99th FCT of intra-datacenter traffic and long-haul traffic by up to 90% and 88%, respectively. As a second observation, DCQCN performs worse in the 600KM testbed, which indicates that the long base-RTT is the key culprit for the performance degradation.

ATC benefits intra-datacenter flows. We repeat the previous evaluation in the 100KM testbed with varying message size of intra-datacenter flows. Figure 14 shows the results. We

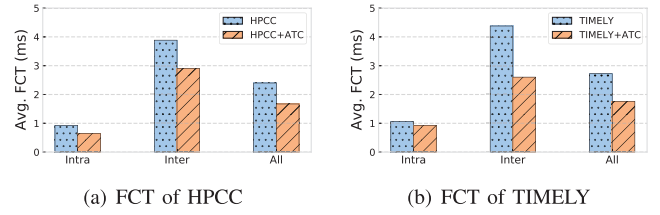


Fig. 15. [Testbed] FCT of different traffic patterns (compare with HPCC and TIMELY).

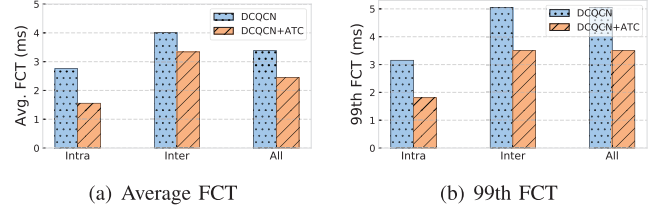


Fig. 16. [Testbed] FCT of different traffic with ON-OFF workload.

can see that ATC improves the average FCT by up to 58% with local congestion scenario. We also notice that ATC performs better with local congestion scenario, which is similar with the simulation results in § III-A. This is because downlink are more likely to be congested and thus the risk of distal congestion is more severe. Nevertheless, ATC can still improve the FCT by up to 56% with distal congestion scenario.

ATC enhances other host-driven methods. We deploy ATC with HPCC and TIMELY and repeat previous evaluations. Figure 15 shows the average FCT of intra-datacenter and inter-datacenter traffic. We can see that the FCTs are improved by 31% and 37% with HPCC and TIMELY, respectively. Online appendix [58] shows more detailed results. The results show that HPCC suffers unfairness and TIMELY cannot react to small long-haul burst traffic quickly since its adjustment factors T_{high} and T_{low} become considerably large and clumsy.

ATC benefits different traffic workload. To evaluate the performance of ATC across various scenarios, we conduct ON-OFF traffic workload and show the results in Figure 16. We can see that ATC can benefit different traffic workload and reduce both the average and 99th FCT of intra-datacenter traffic by 44% and 42%, respectively.

C. Large-Scale Simulations

We compare ATC with different host-driven and switch-driven methods using large-scale NS3 simulations.

1) Compare with host-driven control methods. We show that ATC can ensure fast congestion avoidance and reduce average FCT under different traffic loads or different control method configurations.

Intra-DC traffic dominant scenario. Figure 17 shows the FCT for different traffic patterns. We can see that ATC significantly improves FCTs of both intra-DC and inter-DC traffic. With ATC, the overall average FCTs are reduced by 25%, 31%, 37%, 67%, 75% compared with DCQCN, TIMELY, DCQCN+w, TIMELY+w and HPCC, respectively. As a second observation, we find that the long-haul based

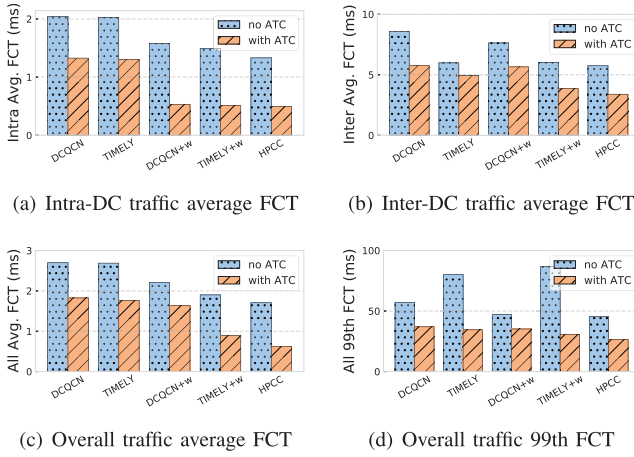


Fig. 17. Intra-DC traffic at 50% load + Inter-DC traffic at 20% load.

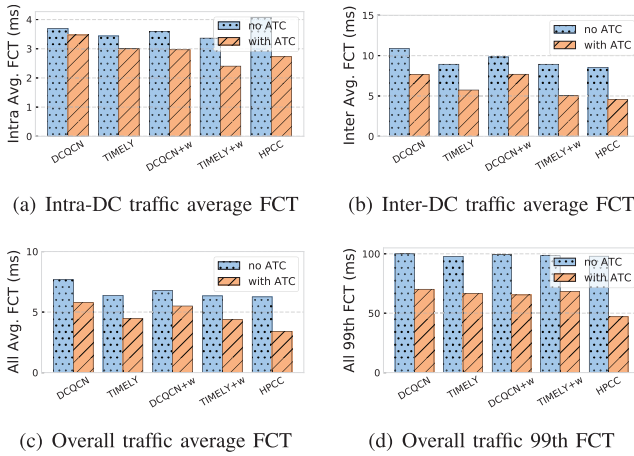


Fig. 18. Intra-DC traffic at 20% load + Inter-DC traffic at 50% load.

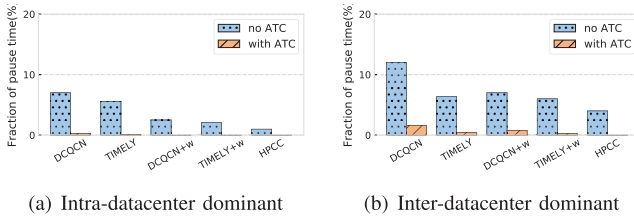


Fig. 19. PFC activation with different traffic scenarios.

sending window does not optimize intra-DC traffic. ATC sets an appropriate window within the datacenter scale. As a result, the intra-DC traffic average FCT is reduced by up to 65%.

Inter-DC traffic dominant scenario. Figure 18 shows the FCT for different traffic patterns. The inter-datacenter traffic faces the HoL problem at the distal DCI-switch and causes a significantly large 99th FCT. ATC leverages VOQs and proactively long-haul schedule scheme to avoid the HoL problem. Thus, the overall 99th FCTs are reduced by 27%, 31%, 33%, 31% and 52% compared with DCQCN, TIMELY, DCQCN+w, TIMELY+w and HPCC, respectively.

ATC ensures fast congestion avoidance. Figure 19 shows the PFC activation time under the previous two traffic loads. We can see that host-driven methods frequently trigger PFC

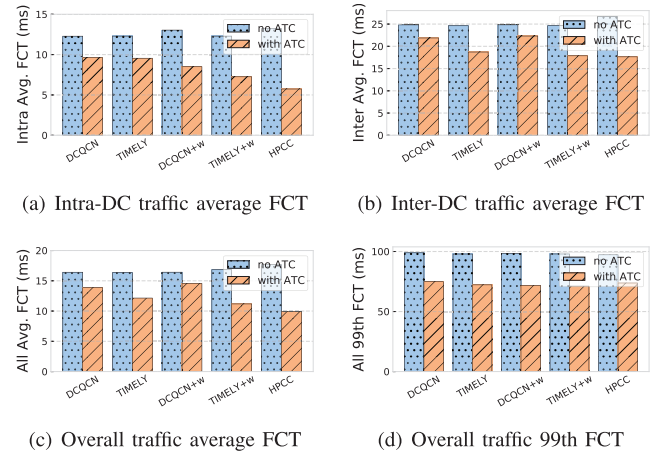


Fig. 20. Different interconnect topology with a 100Gbps long-haul link. Intra-DC traffic at 30% load + Inter-DC traffic at 30% load.

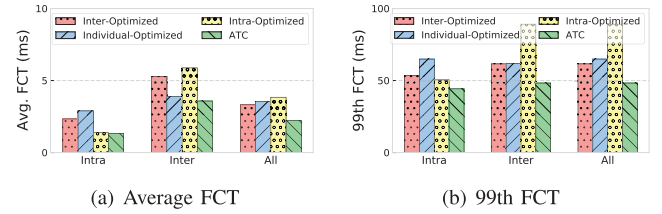


Fig. 21. Different optimized initial windows of HPCC. Intra-DC traffic at 30% load + Inter-DC traffic at 30% load. Inter-Optimized and Intra-Optimized set initial window as long-haul BDP and single-datacenter BDP, respectively. Individual-Optimized sets different initial window for different traffic patterns.

pause frames and the added sending window slightly mitigate this problem. In contrast, ATC shortens the control loop to a single datacenter scale and achieves fast congestion avoidance. Thus, PFC pause frames are reduced by up to 98% with DCQCN and TIMELY, and are reduced to zero with HPCC.

Vary topologies and control method configurations. We conduct experiments with different scenarios to evaluate the adaptability of ATC.

First, we conduct different interconnect topologies by changing the long-haul link rate to 100Gbps. In ATC, the departure feedback returns the corresponding congestion information to the sender, which can quickly avoid congestion. Figure 20 shows the FCTs. We can see that ATC reduces the overall traffic average FCT by up to 44%.

Second, we conduct three types of initial window configurations: intra-optimized, inter-optimized and individual-optimized. Figure 21 shows the FCT for different patterns of traffic with HPCC. We can see that the inter-datacenter traffic suffers performance penalty with intra-optimized configuration due to long-haul bandwidth waste. For intra-datacenter traffic, the performance of individual-optimized configuration is worse than others. This is because the different initial windows bring unfairness between traffic. After congestion occurs, intra-datacenter traffic reduces rate more dramatically than others. In contrast, ATC sets the same initial window to ensure fairness between different traffic.

Third, we conduct evaluations with two DCQCN configurations. Figure 22 shows the FCTs. The progressive Config1,

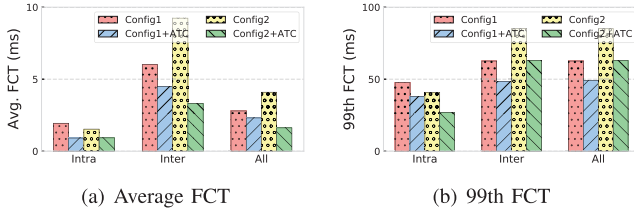


Fig. 22. Different host-driven control method configurations (DCQCN). Intra-DC traffic at 30% load + Inter-DC traffic at 30% load. Config1: $T_i = 55us$, $T_d = 50us$. Config2: $T_i = 300us$, $T_d = 4us$.

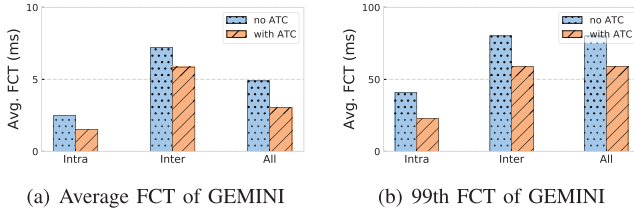


Fig. 23. Different host-driven control method: GEMINI. Intra-DC traffic at 30% load + Inter-DC traffic at 30% load.

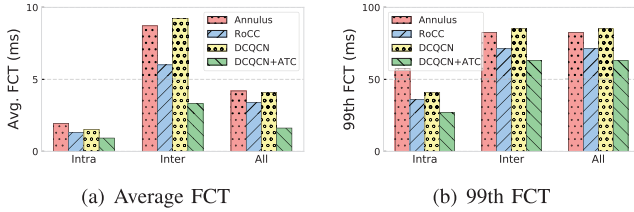


Fig. 24. Different switch-driven methods. Compare ATC with Annulus and RoCC. Intra-DC traffic at 30% load + Inter-DC traffic at 30% load.

$T_i = 55us$ and $T_d = 50us$, is recommended in [9]. The conservative Config2, $T_i = 300us$ and $T_d = 4us$, is the default setting suggested by a major NIC vendor [11]. We can see that the progressive setting benefits the inter-DC traffic and the conservative setting benefits the intra-DC traffic. ATC is compatible with both configurations and can reduce the overall average FCT by up to 51%.

Fourth, we evaluate the performance of GEMINI by comparing it to ATC deployed with DCQCN using a sending window and Figure 23 shows the results. We can see that since GEMINI still maintains the long end-to-end control loop and leverages the inefficiency delay-based congestion signaling for long-haul traffic, it performs worse than ATC. Specifically, ATC can improve the average FCT and 99th FCT by up to 38% and 45%, respectively.

2) Compare with switch-driven control methods. We show that ATC can reduce the 99th FCT with easy deployment complexity.

First, we compare ATC with near-source switch feedback solutions, Annulus and RoCC, in Figure 24. We can see that benefiting from the overall switch deployment, RoCC improves the traffic FCT compared with DCQCN. However, since RoCC tends to overreact and fails to resolve congestion in the distal datacenter, it still performs worse than ATC. Annulus can not deal with complex congestion from multiple ToRs and thus increases the average FCT of intra-datacenter

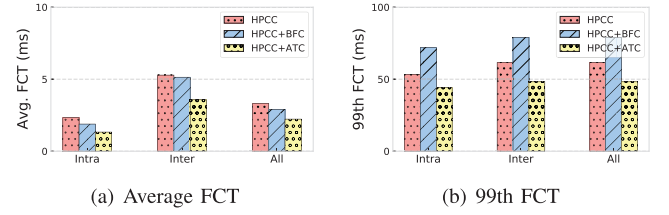


Fig. 25. Different switch-driven methods. Compare ATC with BFC. Intra-DC traffic at 30% load + Inter-DC traffic at 30% load.

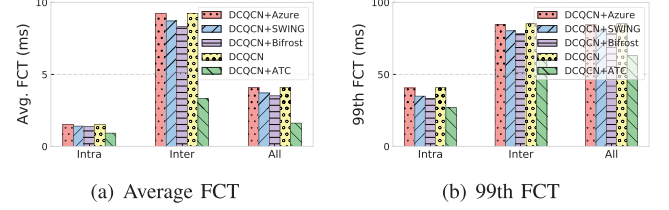


Fig. 26. Different switch-driven methods. Compare ATC with Azure, SWING and Bifrost. Intra-DC traffic at 30%+ Inter-DC traffic at 30% load.

traffic by $1.2\times$ compared with DCQCN. In contrast, ATC deploys at the DCI-switch that collects link load information across the whole datacenter and shows the best performance.

Second, we compare ATC with per-hop switch control solution, BFC, in Figure 25. We can see that BFC degrades the average FCT but increases the 99th FCT. This is because BFC heavily utilizes VOQs and the HoL problem still exists due to the sticky queue assignment mechanism and hash collision problem, which has also been introduced in [41]. In contrast, ATC leverages the proactively schedule transmission in the long-haul link and the maximum number of VOQ usage at this traffic load is only 7. As a result, ATC improves the 99th FCT by 38% compared with BFC.

Third, we compare ATC with other DCI-switch control solutions, Azure, SWING and Bifrost, in Figure 26. We can see that Azure only maintains deep DCI-switch buffers and can not deal with hybrid congestion happened in the local/distal datacenter. Thus, it shows similar results with pure DCQCN. SWING utilizes the PFC relay scheme and can slightly mitigate the congestion in the distal datacenter and Bifrost schedules long-haul link transmission to alleviate buffer usage of the distal DCI-switch. However, SWING and Bifrost still maintains the end-to-end control loop and thus only slightly improves the performance.

D. Design Choices

VOQ usage. We monitor the number of VOQ usage in previous experiments and find that at most 20 VOQs are used when conducting the intra-datacenter traffic dominant scenario. Then, we vary to a larger datacenter scale with 320 servers and corresponding Fat-Tree topology. We reassign the link bandwidth between intra-datacenter to 400Gbps and use four 800Gbps links to connect DCI-switches. The buffer size of intra-datacenter switch and DCI-switch are 32MB and 512MB, respectively. We reproduce the intra-datacenter traffic dominant scenario and monitor the VOQ usage again. At this time, the maximum usage number becomes 58. Note

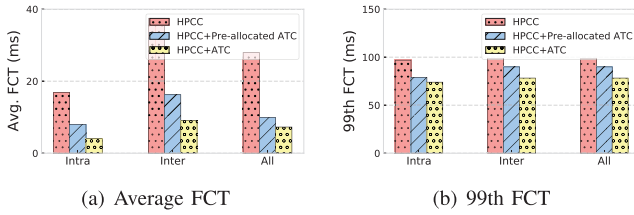


Fig. 27. Different ATC configurations. Intra-DC traffic at 20% load + Inter-DC traffic to random receivers at 10% load + Inter-DC traffic to eight receivers at 40% load.

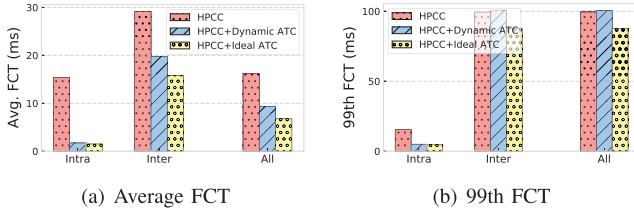


Fig. 28. Different ATC configurations. Intra-DC traffic at 20% load and 50% load + Inter-DC traffic at 30% load + Incast traffic at 10% load. Dynamic ATC represents ATC with long-haul traffic control. Ideal ATC represents ATC without long-haul traffic control.

that the VOQ usage does not increase proportionally with the datacenter scale and the reasons are twofold: (i) larger datacenters have more redundant fabric bandwidth. Thus, it is harder to trigger consistent hybrid congestion in the distal datacenter and VOQs can be released quickly; (ii) in larger datacenters, the local DCI-switch connects with more core switches. Thus, the long-haul traffic tends to create congestion at the local DCI-switch before transmit to the long-haul link, which can be early alleviated by the departure feedback.

Derived from these experiments, we notice that in most cases, the maximum number of VOQ usage is fairly smaller than the end-hosts. As for commodity switch ASIC Tofino2 (with 128 queues), we set the $N_{threshold}$ by 86, which will not be triggered in our experiments. However, we still maintain the long-haul control to avoid corner cases. To make a stress test with the long-haul traffic control mechanism, we set the workload of unidirectional inter-datacenter traffic at 30%, sender-side intra-datacenter traffic at 20% and receiver-side intra-datacenter traffic at 50%. We further add 10% Incast inter-datacenter traffic. We monitor the usage of VOQ and find that the maximum number of VOQ usage reaches 106. We compare ATC with or without long-haul traffic control. For ATC with long-haul traffic control, we choose to evaluate the performance with commodity switch ASIC Tofino2. Thus, we set the total number of VOQ as 128 and $N_{threshold}$ as 86. Figure 28 shows the result. We can see that ATC without long-haul traffic control can significantly improve the average FCT of intra-datacenter traffic by 91% while improving the average FCT of inter-datacenter traffic by 52%. As a second observation, ATC with long-haul traffic control can also benefit intra-datacenter traffic by 88% and perform similarly to ATC without long-haul traffic control. In addition, although ATC with long-haul traffic control compromises the 99th FCT for inter-datacenter traffic, it can still improve the average FCT by 33%

VOQ pre-allocated. We randomly select eight end-hosts as receivers and set the inter-datacenter traffic at 40%. Then, we add the random inter-datacenter traffic at 10% and intra-datacenter traffic at 20%. We compare two ATC configurations: 8 pre-allocated VOQs ATC and dynamically VOQs ATC. Figure 27 shows the result. We can see that the pre-allocated ATC reduces the average FCT of intra-datacenter traffic by 50% and inter-datacenter traffic by 51%. This indicates that the HoL problem is severe in the long-haul transport and a small amount of VOQs can make a great improvement.

VII. DISCUSSION AND FUTURE WORK

Classifying inter-DC and intra-DC traffic is nontrivial.

In today's cloud network, the virtual subnets technique has been widely deployed across DCs to facilitate management. This technique decouples the IPs from the DC locations [39]. Since RDMA implements the transport functionalities (e.g., congestion control logic) entirely in RNIC hardware, maintaining an always-up-to-date IP-to-DC mapping is infeasible for the limited on-chip SRAM. This awkward method also forgoes the management benefit of using virtual subnets. In addition, even if we pay the price for adopting this method, the throughput unfairness between intra and inter datacenter traffic still degrades the performance.

Long-haul loss recovery requirements. Studies on loss recovery are vast, but they fail to be compatible with the characteristics of long-haul communications (e.g., large base-RTT and random loss). Some switch-driven mechanisms protect important packets and eliminate timeouts. For example, TLT [67] marks important packets at end-hosts and selectively drops unimportant ones using a color-aware dropping method. Lightning [68] enhances the switch to send loss notifications directly to the senders. However, the sender still suffers a long feedback loop when losses occur in the long-haul link or distal datacenter. Besides, Mellanox ConnectX RNIC implements the lossy-RoCE acceleration, including limited sending windows and optimizations on congestion control such as slow restarts. However, the NACKs caused by random loss may incur spurious flow rate reduction. In this paper, ATC mainly focuses on rate control and leave the long-haul link loss recovery to the future work.

RNIC hardware limitations on long-haul performance. Commercial RNICs usually take a DRAM-free architecture to reduce cost, power consumption and area. Thus, RNICs can cache only a small number of Queue Pairs (QP) and Work Queue Element (WQE) on the chip. The long-haul transport frequently consumes large on-chip memory due to the long base RTT, which leads to frequent cache (e.g., WQE cache) misses and significant performance degradation. Recent work SRNIC [34] minimizes most on-chip data structures by proposing a cache-free QP scheduler and memory-free selective repeat method. These works are orthogonal and complementary to ATC. And we believe the development of RNIC architecture will enhance the performance of cross-datacenter RDMA networks, including high scalability and fast loss recovery at scale.

Programmable switch improvement. ATC demands efficient interaction and high workload handling between the control plane and data plane. However, state-of-the-art programmable switches faces limitations in this interaction, primarily due to constraints such as low-performance switch CPU, the slow PCIe between the switching ASIC and switch CPU and the limited memory in the switching ASIC. Recent work [69], [70], [71] try to improve the performance of packet processing using external devices such as CPUs or FPGAs and achieve a Tbps throughput. These works are orthogonal to ATC and we believe the development of programmable switches will improve the performance of ATC, including fast reaction and high scalability at scale. In addition, we are collaborating with one switch vendor to implement ATC to the switch hardware.

VIII. RELATED WORK

Recently, tons of traffic control methods have been proposed. We have discussed the closely related schemes (e.g., Azure [1], BFC [14], Annulus [12]) in § III. Here, we only overview some other ideas which have not been discussed elsewhere.

Receiver-driven rate control. A recent line of works pHost [72], NDP [73], ExpressPass [74] and Homa [75] aiming to proactively allocate link bandwidth using receiver-driven credits. Specifically, pHost and Homa assume that the network core is free of congestion. However, the congestion can easily happen at DCI-switches due to the oversubscribed datacenter interconnect topology. ExpressPass and NDP require overall switch modifications or advanced network support (e.g., cutting payload [76]). In addition, Aeolus [77] alleviates the “first-RTT problem” by selectively dropping packets. However, it will suffer sluggish loss detection and cause severe performance drop in the long-haul RDMA network. To avoid the above problems and ease deployment with commercial RNICs, ATC chooses to be cooperate with sender-driven rate control methods.

Packet scheduling scheme. To achieve near-optimal latency, pFabric [78] requires fine-grained priority scheduling logics at switches and correct flow size information at end-hosts, which are hard to deploy [73]. PIAS [79] eliminates the advanced knowledge of flow size using multi-level priority queues in the switches. QJUMP [80] manually specified priorities on a per-application basis. These scheduling schemes are primarily orthogonal to the rate control. Recent work Bolt [81] monitors the queue length and port link utilization across all switches to assess network load information and generate feedback to the sender improve network utilization. SFC [82] also monitors the queue length across all switches and calculates a pause duration back to the sender to alleviate queue buildup. However, both of these schemes require deployment across all switches, whereas ATC only involves modifications to DCI-switches.

WAN transport. Traditional loss-based WAN transport protocols (e.g., Cubic [83]) suffer high packet loss and deep queueing delay. To avoid these problems, subsequent protocols (e.g., Vegas [84], Copa [85], BBR [86]) leverage delay signal to detect congestion. However, these methods consider the

WAN scenario only and will suppress the performance of intra-datacenter traffic. ATC is compatible with these delay-based TCP mechanisms, using the same deployment approach as with delay-based RDMA mechanisms by generating delay signals directly from the DCI-switch. We will combine them in the future.

IX. CONCLUSION

This paper re-architects traffic control for cross-datacenter RDMA networks and presents ATC, a readily deployable solution focusing on alleviating hybrid traffic congestion, as a building block for existing host-driven methods. By deploying on two-side DCI-switches, ATC can shorten the control loop to the scale of a single datacenter and aggregate congestion information across the whole datacenter. We implement ATC on commodity P4-based switches and evaluate it using real-world testbed experiments and large-scale simulations. The extensive evaluations confirm that ATC is a promising substrate for strengthening the performance of cross-datacenter RDMA networks. We believe ATC is a start towards traffic control for future high-speed cross-datacenter RDMA networks. *This work does not raise any ethical issues.*

REFERENCES

- [1] B. Wei et al., “Empowering Azure storage with RDMA,” in *Proc. 20th USENIX Symp. Netw. Syst. Design Implement. (NSDI)*, 2023, pp. 49–67.
- [2] K. Hsieh et al., “Gaia: Geo-distributed machine learning approaching LAN speeds,” in *Proc. NSDI*, Mar. 2017, pp. 629–647.
- [3] Y. Gao et al., “When cloud storage meets RDMA,” in *Proc. NSDI*, Jan. 2021, pp. 519–533.
- [4] X. Zhou and H. Liu, “Pluggable DWDM: Considerations for campus and metro DCI applications,” in *Proc. Eur. Conf. Opt. Commun.*, 2016, pp. 1–13.
- [5] Y. Sverdlik, “Facebook rethinks in-region data center interconnection,” DataCenter Knowl., 2018. [Online]. Available: <https://www.datacenterknowledge.com/networking/facebook-rethinks-in-region-data-center-interconnection>
- [6] S. Alam, P. Agnihotri, and G. Dumont, “AWS re: Invent. Enterprise fundamentals: Design your account and VPC architecture for enterprise operating models,” 2016. [Online]. Available: https://www.youtube.com/watch?v=Ja_zcw74gfE&ab_channel=AmazonWebServicesShahbaz
- [7] A. Singh et al., “Jupiter rising: A decade of clos topologies and centralized control in Google’s datacenter network,” *ACM SIGCOMM Comput. Commun. Rev.*, vol. 45, no. 4, pp. 183–197, 2015.
- [8] V. Dukic et al., “Beyond the mega-data center: Networking multi-data center regions,” in *Proc. Annu. Conf. ACM Special Interest Group Data Commun. Appl. Technol. Archit. Protocols Comput. Commun.*, 2020, pp. 765–781.
- [9] Y. Zhu et al., “Congestion control for large-scale RDMA deployments,” *ACM SIGCOMM Comput. Commun. Rev.*, vol. 45, no. 4, pp. 523–536, Sep. 2015.
- [10] R. Mittal et al., “TIMELY: RTT-based congestion control for the datacenter,” *ACM SIGCOMM Computer Communication Review*, vol. 45, no. 4, pp. 537–550, 2015.
- [11] Y. Li et al., “HPCC: High precision congestion control,” in *Proc. ACM Special Interest Group Data Commun.*, Aug. 2019, pp. 44–58.
- [12] A. Saeed et al., “Annulus: A dual congestion control loop for datacenter and WAN traffic aggregates,” in *Proc. Annu. Conf. ACM Special Interest Group Data Commun. Appl. Technol. Archit. Protocols Comput. Commun.*, Jul. 2020, pp. 735–749.
- [13] P. Taheri, D. Menikkumbura, E. Vanini, S. Fahmy, P. Eugster, and T. Edsall, “RoCC: Robust congestion control for RDMA,” in *Proc. 16th Int. Conf. Emerg. Netw. Exp. Technol.*, Nov. 2020, pp. 17–30.
- [14] P. Goyal, P. Shah, N. K. Sharma, M. Alizadeh, and T. E. Anderson, “Backpressure flow control,” in *Proc. Workshop Buffer Sizing*, Dec. 2019, pp. 1–3.
- [15] Y. Chen et al., “Swing: Providing long-range lossless RDMA via PFC-relay,” *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 1, pp. 63–75, Jan. 2023.

- [16] Cisco Global Cloud Index 2015–2020. Accessed: Nov. 2016. [Online]. Available: <http://www.cisco.com/c/dam/en/us/solutions/collateral/service-provider/global-cloud-index-gci/white-paper-c11-738085.pdf>
- [17] Y. Feng, B. Li, and B. Li, “Jetway: Minimizing costs on inter-datacenter video traffic,” in *Proc. 20th ACM Int. Conf. Multimedia*, Oct. 2012, pp. 259–268.
- [18] Y. Lin, H. Shen, and L. Chen, “EcoFlow: An economical and deadline-driven inter-datacenter video flow scheduling system,” in *Proc. IEEE 35th Int. Conf. Distrib. Comput. Syst.*, Jun. 2015, pp. 736–737.
- [19] What is a Hybrid Cloud? — Google Cloud. Accessed: 2025. [Online]. Available: <https://cloud.google.com/learn/what-is-hybrid-cloud/#section-1>
- [20] What is Hybrid Cloud Computing — Microsoft Azure. Accessed: 2025. [Online]. Available: <https://azure.microsoft.com/en-us/resources/cloud-computing-dictionary/what-is-hybrid-cloud-computing/>
- [21] J. Yao, P. Lu, L. Gong, and Z. Zhu, “On fast and coordinated data backup in geo-distributed optical inter-datacenter networks,” *J. Lightw. Technol.*, vol. 33, no. 14, pp. 3005–3015, Jul. 15, 2015.
- [22] C. Devellder, J. Buysse, B. Dhoedt, and B. Jaumard, “Joint dimensioning of server and network infrastructure for resilient optical Grids/Clouds,” *IEEE/ACM Trans. Netw.*, vol. 22, no. 5, pp. 1591–1606, Oct. 2014.
- [23] Oracle White Paper: Backup is Not Archiving: Reduce TCO and Improve Data Protection With a Customized Archive Solution. Accessed: 2013. [Online]. Available: <http://www.oracle.com/us/products/servers-storage/storage/tape-storage/backupnotarchivefinal-120513gc-2083314.pdf>
- [24] M. F. Habib, M. Tornatore, M. De Leenheer, F. Dikbiyik, and B. Mukherjee, “Design of disaster-resilient optical datacenter networks,” *J. Lightw. Technol.*, vol. 30, no. 16, pp. 2563–2573, Aug. 2012. [Online]. Available: <https://ieeexplore.ieee.org/document/6208805>
- [25] S. Kandula, I. Menache, R. Schwartz, and S. R. Babbula, “Calendar for wide area networks,” in *Proc. ACM Conf. SIGCOMM*, Aug. 2014, pp. 515–526.
- [26] S. Jain et al., “B4: Experience with a globally-deployed software defined WAN,” *ACM SIGCOMM Comput. Commun. Rev.*, vol. 43, no. 4, pp. 3–14, 2013.
- [27] How Microsoft Builds Its Fast and Reliable Global Network. Accessed: Mar. 15, 2017. [Online]. Available: <https://azure.microsoft.com/en-us/blog/how-microsoft-builds-its-fast-and-reliable-global-network/>
- [28] Building Express Backbone: Facebook’s New Long-Haul Network. Accessed: May 1, 2017. [Online]. Available: <https://engineering.fb.com/2017/05/01/data-center-engineering/building-express-backbone-facebook-s-new-long-haul-network/>
- [29] M. Filer, J. Gaudette, Y. Yin, D. Billor, Z. Bakhtiari, and J. L. Cox, “Low-margin optical networking at cloud scale [Invited],” *J. Opt. Commun. Netw.*, vol. 11, no. 10, p. C94, 2019.
- [30] Data Center Interconnect — What is It? See Solutions From Viavi. Accessed: 2025. [Online]. Available: <https://www.viavisolutions.com/en-us/resources/learning-center/what-data-center-interconnect>
- [31] What is an Optical Module. Accessed: Oct. 10, 2020. [Online]. Available: <https://support.huawei.com/enterprise/en/doc/EDOC1100130737>
- [32] Infiniband Architecture Volume 1, General Specifications, Release 1.2.1. [Online]. Available: <http://www.infinibandta.org/specs>
- [33] C. Guo et al., “RDMA over commodity Ethernet at scale,” in *Proc. ACM SIGCOMM Conf.*, Aug. 2016, pp. 202–215.
- [34] Z. Wang et al., “SRNIC: A scalable architecture for RDMA NICs,” in *Proc. 20th USENIX Symp. Netw. Syst. Design Implement. (NSDI)*, 2023, pp. 1–14.
- [35] W. Cheng, K. Qian, W. Jiang, T. Zhang, and F. Ren, “Re-architecting congestion management in lossless Ethernet,” in *Proc. NSDI*, Jan. 2020, pp. 19–36.
- [36] Priority-Based Flow Control, IEEE Standard 802.1Qbb, 2011. Accessed: May 12, 2019. [Online]. Available: <http://www.ieee802.org/1/pages/802.1bb.html>
- [37] S. Hu et al., “Deadlocks in datacenter networks: Why do they form, and how to avoid them,” in *Proc. 15th ACM Workshop Hot Topics Netw.*, Nov. 2016, pp. 92–98.
- [38] G. Kumar et al., “Swift: Delay is simple and effective for congestion control in the datacenter,” in *Proc. Annu. Conf. ACM Special Interest Group Data Commun. Appl., Technol., Archit., Protocols Comput. Commun.*, Jul. 2020, pp. 514–528.
- [39] G. Zeng et al., “Congestion control for cross-datacenter networks,” in *Proc. IEEE 27th Int. Conf. Netw. Protocols (ICNP)*, Oct. 2019, pp. 1–12.
- [40] 802.1Qau—Congestion Notification, 2009. Accessed: Dec. 15, 2009. [Online]. Available: <https://1.ieee802.org/dcb/802-1qau/>
- [41] K. Liu et al., “Floodgate: Taming incast in datacenter networks,” in *Proc. 17th Int. Conf. Emerg. Netw. Exp. Technol.*, Dec. 2021, pp. 30–44.
- [42] F. Cangialosi, A. Narayan, P. Goyal, R. Mittal, M. Alizadeh, and H. Balakrishnan, “Site-to-site Internet traffic control,” in *Proc. 16th Eur. Conf. Comput. Syst.*, Apr. 2021, pp. 574–589.
- [43] P. Yu et al., “Bifrost: Extending RoCE for long distance inter-DC links,” in *Proc. IEEE 31st Int. Conf. Netw. Protocols (ICNP)*, Oct. 2023, pp. 1–12.
- [44] R. Mittal et al., “Revisiting network support for RDMA,” in *Proc. Conf. ACM Special Interest Group Data Commun.*, Aug. 2018, pp. 313–326.
- [45] Y. Lu et al., “Memory efficient loss recovery for hardware-based transport in datacenter,” in *Proc. 1st Asia-Pacific Workshop Netw.*, Aug. 2017, pp. 22–28.
- [46] R. Miao et al., “From Luna to solar: The evolutions of the compute-to-storage networks in Alibaba cloud,” in *Proc. ACM SIGCOMM Conf.*, Aug. 2022, pp. 753–766.
- [47] G. Chen et al., “MP-RDMA: Enabling RDMA with multi-path transport in datacenters,” *IEEE/ACM Trans. Netw.*, vol. 27, no. 6, pp. 2308–2323, Dec. 2019.
- [48] X. Wang et al., “StaR: Breaking the scalability limit for RDMA,” in *Proc. IEEE 29th Int. Conf. Netw. Protocols (ICNP)*, Nov. 2021, pp. 1–11.
- [49] Q. Li et al., “Flor: An open high performance RDMA framework over heterogeneous RNICs,” in *Proc. 17th USENIX Symp. Oper. Syst. Design Implement. (OSDI)*, 2023, pp. 931–948.
- [50] Broadcom. (2021). BCM88800 Traffic Management Architecture. Accessed: Feb. 19, 2021. [Online]. Available: <https://docs.broadcom.com/doc/88800-DG1-PUB>
- [51] Cisco. (2010). Cisco Nexus 5548P Switch Architecture. Accessed: Sep. 23, 2010. [Online]. Available: https://www.cisco.com/c/en-us/products/collateral/switches/nexus-5548p-switch/white_paper_c11-622479.html
- [52] Huawei. (2021). CloudEngine 12800 Series Data Center Core Switches. [Online]. Available: <https://carrier.huawei.com/en/products/fixed-network/b2b/ethernet-switches/dc-switches/ce12800>
- [53] Juniper. (2017). Understanding CoS Virtual Output Queues (VOQs) on QFX10000 Switches. [Online]. Available: https://www.juniper.net/documentation/en_US/junos/topics/concept/cos-qfx-series-voq-understanding.html
- [54] Broadcom. BCM88690, StrataDNX 10 Tb/s Scalable Switching Device, Product Brief. Accessed: Apr. 6, 2020. [Online]. Available: <https://www.arista.com/assets/data/pdf/Whitepapers/Arista7500RSwitchArchitectureWP.pdf>
- [55] Cisco Silicon One. Accessed: Apr. 2, 2025. [Online]. Available: <https://www.cisco.com/c/en-us/solutions/silicon-one.html>
- [56] N. Zilberman, G. Bracha, and G. Schukin, “Stardust: Divide and conquer in the data center network,” in *Proc. NSDI*, Feb. 2019, pp. 141–159.
- [57] J. Perry, A. Ousterhout, H. Balakrishnan, D. Shah, and H. Fugal, “Fastpass: A centralized ‘zero-queue’ datacenter network,” in *Proc. ACM Conf. SIGCOMM*, Aug. 2014, pp. 307–318.
- [58] (2024). ATC Appendix. [Online]. Available: <https://anonymous.4open.science/r/ATCC-appendix-C210/Appendix.pdf>
- [59] 2021 P4 Workshop. Accessed: May 18, 2021. [Online]. Available: <https://opennetworking.org/wp-content/uploads/2021/05/2021-P4-WS-Vladimir-Gurevich-Slides.pdf>
- [60] W. Mansour, N. Janvier, and P. Fajardo, “FPGA implementation of RDMA-based data acquisition system over 100 GbE,” 2018, *arXiv:1806.08939*.
- [61] Scapy/roce.py At Master. Accessed: 2025. [Online]. Available: <https://github.com/secdev/scapy/blob/master/scapy/contrib/roce.py#L153-L183>
- [62] Y. Shpigelman, I. Burstein, N. Bloch, R. Zuck, and R. Moyal, “Programmable congestion control communication scheme,” U.S. Patent 11,296,988, Apr. 5, 2022.
- [63] NVIDIA BLUEFIELD-3 DPU. Accessed: 2024. [Online]. Available: <https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/documents/datasheet-nvidia-bluefield-3-dpu.pdf>
- [64] M. Al-Fares, A. Loukissas, and A. Vahdat, “A scalable, commodity data center network architecture,” *ACM SIGCOMM Comput. Commun. Rev.*, vol. 38, no. 4, pp. 63–74, Oct. 2008.
- [65] High-Precision-Congestion-Control/Simulation/Mix/Config.txt at Master. Accessed: 2025. [Online]. Available: <https://github.com/alibaba-edu/High-Precision-Congestion-Control/blob/master/simulation/mix/config.txt>
- [66] High Precision Congestion Control Trafficgen at Master. Accessed: 2025. [Online]. Available: https://github.com/alibaba-edu/High-Precision-Congestion-Control/tree/master/traffic_gen
- [67] H. Lim, W. Bai, Y. Zhu, Y. Jung, and D. Han, “Towards timeout-less transport in commodity datacenter networks,” in *Proc. 16th Eur. Conf. Comput. Syst.*, Apr. 2021, pp. 33–48.

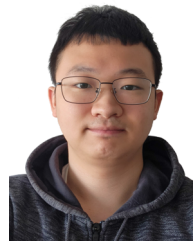
- [68] Q. Meng and F. Ren, "Lightning: A practical building block for RDMA transport control," in *Proc. IEEE/ACM 29th Int. Symp. Quality Service (IWQOS)*, Jun. 2021, pp. 1–10.
- [69] C. Zeng et al., "Tiara: A scalable and efficient hardware acceleration architecture for stateful layer-4 load balancing," in *Proc. 19th USENIX Symp. Netw. Syst. Design Implement. (NSDI)*, 2022, pp. 1345–1358.
- [70] L. Zeno, D. R. K. Ports, J. Nelson, and M. Silberstein, "SwiShmem: Distributed shared state abstractions for programmable switches," in *Proc. 19th ACM Workshop Hot Topics Netw.*, Nov. 2020, pp. 160–167.
- [71] M. Scazzariello, T. Caiazz, H. Ghasemirahni, T. Barbette, D. Kostić, and M. Chiesa, "A high-speed stateful packet processing approach for Tbps programmable switches," in *Proc. 20th USENIX Symp. Netw. Syst. Design Implement. (NSDI)*, 2023, pp. 1237–1255.
- [72] P. X. Gao, A. Narayan, G. Kumar, R. Agarwal, S. Ratnasamy, and S. Shenker, "PHost: Distributed near-optimal datacenter transport over commodity network fabric," in *Proc. 11th ACM Conf. Emerg. Netw. Exp. Technol.*, Dec. 2015, pp. 1–12.
- [73] M. Handley et al., "Re-architecting datacenter networks and stacks for low latency and high performance," in *Proc. Conf. ACM Special Interest Group Data Commun.*, Aug. 2017, pp. 29–42.
- [74] I. Cho, K. Jang, and D. Han, "Credit-scheduled delay-bounded congestion control for datacenters," in *Proc. Conf. ACM Special Interest Group Data Commun.*, Aug. 2017, pp. 239–252.
- [75] B. Montazeri, Y. Li, M. Alizadeh, and J. Ousterhout, "Homa: A receiver-driven low-latency transport protocol using network priorities," in *Proc. Conf. ACM Special Interest Group Data Commun.*, Aug. 2018, pp. 221–235.
- [76] P. Cheng, F. Ren, R. Shu, and C. Lin, "Catch the whole lot in an action: Rapid precise packet loss notification in data center," in *Proc. 11th USENIX Symp. Netw. Syst. Design Implement. (NSDI)*, 2014, pp. 17–28.
- [77] S. Hu et al., "Aeolus: A building block for proactive transport in datacenters," in *Proc. Annu. Conf. ACM Special Interest Group Data Commun. Appl., Technol., Archit., Protocols Comput. Commun.*, Jul. 2020, pp. 422–434.
- [78] M. Alizadeh et al., "pFabric: Minimal near-optimal datacenter transport," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 43, no. 4, pp. 435–446, 2013.
- [79] W. Bai, L. Chen, K. Chen, D. Han, C. Tian, and H. Wang, "Information-agnostic flow scheduling for commodity data centers," in *Proc. NSDI*, May 2015, pp. 455–468.
- [80] M. P. Grosvenor et al., "Queues don't matter when you can JUMP them!" in *Proc. 12th USENIX Symp. Netw. Syst. Design Implement. (NSDI)*, 2015, pp. 1–14.
- [81] S. Arslan, Y. Li, G. Kumar, and N. Dukkipati, "Bolt: Sub-RTT congestion control for ultra-low latency," in *Proc. 20th USENIX Symp. Netw. Syst. Design Implement. (NSDI)*, 2023, pp. 219–236.
- [82] Y. Le et al., "SFC: Near-source congestion signaling and flow control," 2023, *arXiv:2305.00538*.
- [83] S. Ha, I. Rhee, and L. Xu, "CUBIC: A new TCP-friendly high-speed TCP variant," *ACM SIGOPS Operating Syst. Rev.*, vol. 42, no. 5, pp. 64–74, Jul. 2008.
- [84] L. S. Brakmo, S. W. O'Malley, and L. L. Peterson, "TCP Vegas: New techniques for congestion detection and avoidance," in *Proc. Conf. Commun. Archit., Protocols Appl.*, Oct. 1994, pp. 24–35.
- [85] V. Arun and H. Balakrishnan, "Copa: Practical delay-based congestion control for the internet," in *Proc. 15th USENIX Symp. Netw. Syst. Design Implement. (NSDI)*, 2018, pp. 329–342.
- [86] N. Cardwell, Y. Cheng, C. S. Gunn, S. H. Yeganeh, and V. Jacobson, "BBR: Congestion-based congestion control," *Commun. ACM*, vol. 60, no. 2, pp. 58–66, 2017.



Jiao Zhang received the bachelor's degree from the School of Computer Science and Technology, Beijing University of Posts and Telecommunications (BUPT), China, in July 2008, and the Ph.D. degree from the Department of Computer Science and Technology, Tsinghua University, China, in July 2014. From August 2012 to August 2013, she was a Visiting Student with the Networking Group of ICSI, UC Berkeley. She is currently a Professor with the School of Information and Communication Engineering, BUPT. Her research interests include data center networking and transport layer network protocols.



Yuzhen Su received the bachelor's degree from the School of Electronic and Information Engineering, Beijing Jiaotong University, China, in 2021, and the master's degree from the State Key Laboratory of Network and Switching Technology, Beijing University of Posts and Telecommunications, in 2024. He is currently with China Telecom Research Institute, Beijing, China, as a Network Engineer. His current research interests include data center RDMA networks and transport protocols.



Haoyu Pan received the bachelor's degree from the School of Information and Communication Engineering, Beijing University of Posts and Telecommunications, China, where he is currently pursuing the master's degree with the State Key Laboratory of Networking and Switching Technology. His current research interests include data center networking and software-defined networking.



Mingxuan Yu received the bachelor's degree from the School of Electronic and Information Engineering, Beijing Jiaotong University, China, where he is currently pursuing the master's degree with the State Key Laboratory of Networking and Switching Technology. His current research interests include data center networking and software-defined networking.



Yuan Li received the B.Eng. degree in communication engineering from Beijing University of Posts and Telecommunications (BUPT), Beijing, China, in 2023, where he is currently pursuing the M.S. degree with the State Key Laboratory of Networking and Switching Technology. His research interests include data center networks and network transmission control.



Zirui Wan received the bachelor's degree from the School of Information and Communication Engineering, Beijing University of Posts and Telecommunications, China, in 2020, where he is currently pursuing the Ph.D. degree with the State Key Laboratory of Networking and Switching Technology. His current research interests include data center networking and transport protocols.



Tao Huang (Senior Member, IEEE) received the B.S. degree in communication engineering from Nankai University, Tianjin, China, in 2002, and the M.S. and Ph.D. degrees in communication and information system from Beijing University of Posts and Telecommunications (BUPT), Beijing, China, in 2004 and 2007, respectively. He is currently a Professor with BUPT. His current research interests include network architecture, routing and forwarding, and network virtualization.